



Universidad de las Fuerzas Armadas ESPE

Desarrollo de Software Seguro
Parcial II

Informe técnico del proyecto SecureDataMining DevSecOps

Integrantes:

Cesar Loor
Gabriel Murrillo
Camilo Orrico

Sangolquí, Ecuador
17 de junio de 2026

Resumen

El proyecto implementa un flujo DevSecOps para clasificar cambios de código como *SAFE* o *VULNERABLE* sin usar modelos LLM. La solución combina limpieza de datasets, entrenamiento de un modelo clásico de minería de datos, análisis de Pull Requests, notificaciones por Telegram, bloqueo de merge ante vulnerabilidades, ejecución de pruebas y despliegue automatizado. El notebook completo fue ajustado para entrenar con todos los registros disponibles, usando una arquitectura escalable basada en hashing y clasificación lineal.

1. Objetivo

Construir un pipeline seguro que analice código modificado en Pull Requests, detecte patrones vulnerables y automatice acciones de seguridad: comentario en PR, etiquetado, creación de issue, notificación por Telegram, pruebas, merge y despliegue. Esta aproximación se alinea con prácticas DevSecOps y con la automatización soportada por GitHub Actions [4].

2. Trabajo realizado

Las actividades principales fueron:

- Limpieza y unificación de datasets de vulnerabilidades: CodeXGLUE, D2A, ReVeal, MegaVul, CVEFixes y OWASP 2025.
- Segmentación por lenguaje mediante la columna `language`.
- Entrenamiento del notebook completo con todos los registros: **389313** muestras limpias.
- Uso de `HashingVectorizer` y `SGDClassifier` para escalar el entrenamiento sin construir vocabularios gigantes [2, 3].
- Implementación de análisis de PR con reglas, probabilidad del modelo y reporte JSON.
- Configuración de Telegram por secrets: `TELEGRAM_BOT_TOKEN` y `TELEGRAM_CHAT_ID`, usando la API HTTP oficial [5].
- Actualización de workflows de GitHub Actions para lint, tests, escaneo, bloqueo y deploy.

3. Arquitectura de la solución

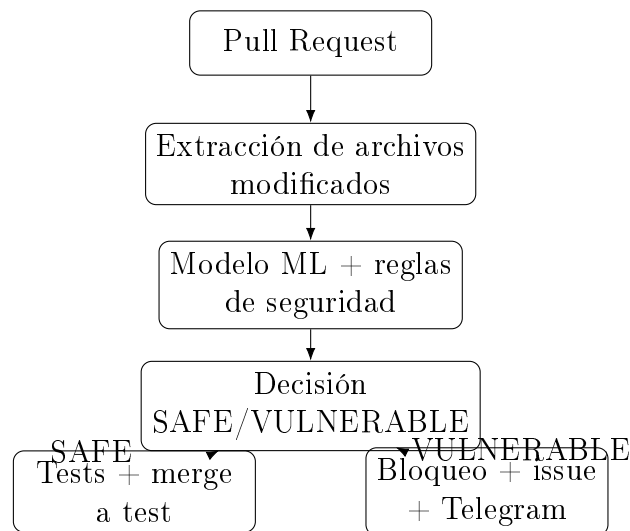


Figura 1: Flujo DevSecOps implementado para revisión segura de Pull Requests.

4. Fragmentos relevantes

4.1. Modelo escalable del notebook completo

```
features = ColumnTransformer([
    ('word_hash',
     HashingVectorizer(n_features=2**18,
                      ngram_range=(1, 2),
                      alternate_sign=False,
                      norm='l2'),
     'raw_code'),
    ('category',
     OneHotEncoder(handle_unknown='ignore', sparse_output=True),
     ['language', 'source']),
    ('numeric',
     StandardScaler(with_mean=False),
     ['code_length', 'line_count']),
])
```

4.2. Regla de bloqueo en Pull Request

```
rule_vulnerable = bool(
    vulnerability_types or cwe_ids or feature_summary.
    suspicious_patterns
)
is_vulnerable = rule_vulnerable or probability >= threshold
```

5. Resultados

El notebook completo se ejecutó con todos los registros disponibles. La Tabla 1 resume las métricas principales.

Cuadro 1: Métricas del entrenamiento completo

Métrica	Valor
Registros limpios	389313
Registros de entrenamiento	291563
Registros de prueba	97334
Accuracy	0.9205
ROC AUC	0.8444
Precision clase vulnerable	0.4769
Recall clase vulnerable	0.3048
F1 clase vulnerable	0.3719

Aunque el accuracy supera el 80 %, se reportó también el *baseline* de clase mayoritaria (0.9228), ya que el dataset completo está desbalanceado. Por ello, ROC AUC, recall y F1 de la clase vulnerable son métricas más informativas que accuracy por sí sola.

6. Conclusiones

Se dejó un repositorio funcional con notebooks reproducibles, pipeline CI/CD seguro y análisis automatizado de Pull Requests. El entrenamiento completo ya usa todos los registros disponibles y el pipeline puede bloquear código vulnerable identificado por reglas o por alta probabilidad del modelo. Como mejora futura, se recomienda entrenar modelos especializados por lenguaje, en especial para C/C++ y Java, para mejorar el recall de vulnerabilidades sin aumentar falsos positivos.

Referencias

- [1] Universidad de las Fuerzas Armadas ESPE, “Sitio oficial ESPE,” 2026. [En línea]. Disponible: <https://www.espe.edu.ec/>. [Accedido: 18-jun-2026].
- [2] scikit-learn developers, “HashingVectorizer,” *scikit-learn documentation*. [En línea]. Disponible: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.HashingVectorizer.html. [Accedido: 18-jun-2026].
- [3] scikit-learn developers, “SGDClassifier,” *scikit-learn documentation*. [En línea]. Disponible: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.SGDClassifier.html. [Accedido: 18-jun-2026].
- [4] GitHub Docs, “GitHub Actions documentation.” [En línea]. Disponible: <https://docs.github.com/actions>. [Accedido: 18-jun-2026].
- [5] Telegram, “Telegram Bot API.” [En línea]. Disponible: <https://core.telegram.org/bots/api>. [Accedido: 18-jun-2026].

- [6] OWASP Foundation, “OWASP Top 10.” [En línea]. Disponible: <https://owasp.org/www-project-top-ten/>. [Accedido: 18-jun-2026].